ABINYATA T
25MDT1163

EDA ACTIVITY 3

1. **Boxplot on the data frame**

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from scipy import stats
from scipy.spatial.distance import mahalanobis
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM


#Box plot
# 1. Create a data frame
# 2. Draw boxplot for any specific column
# 3.identify and eliminate thresold and outliers

data = {
    "A": np.random.normal(50, 10, 100),
    "B": np.random.normal(30, 5, 100)
}

df = pd.DataFrame(data)

# Add extreme outliers
df.loc[95:99, "A"] = [120, 130, 140, 150, 160]
df.loc[95:99, "B"] = [80, 85, 90, 95, 100]

print("Original Shape:", df.shape)
```

ABINYATA T
25MDT1163

```python
# 2. Box Plot Method (IQR)
plt.figure(figsize=(6,4))
sns.boxplot(x=df["A"])
plt.title("Box Plot - Column A")
plt.show()

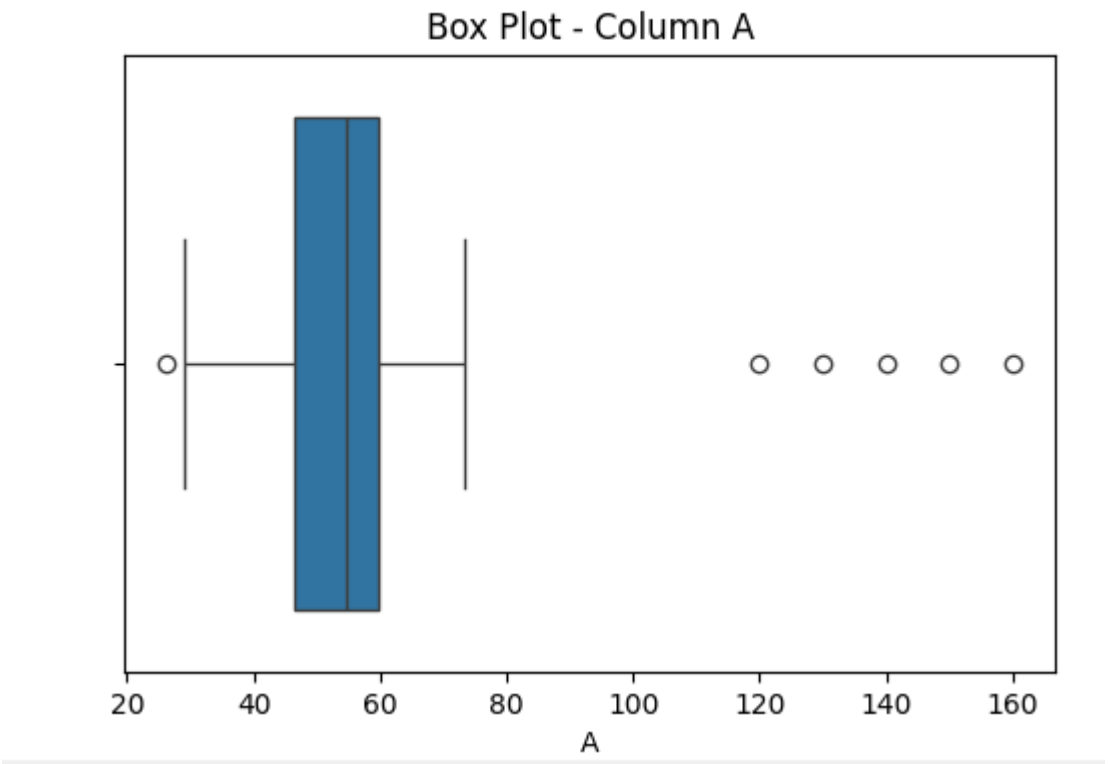
Q1 = df["A"].quantile(0.25)
Q3 = df["A"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_iqr = df[(df["A"] >= lower) & (df["A"] <= upper)]
print("After IQR Removal:", df_iqr.shape)
```

Output:

```
(.venv) PS C:\Users\student\Dc
Original Shape: (100, 2)
```



ABINYATA T
25MDT1163

2. **Normal Distribution**

```
# normal distribution

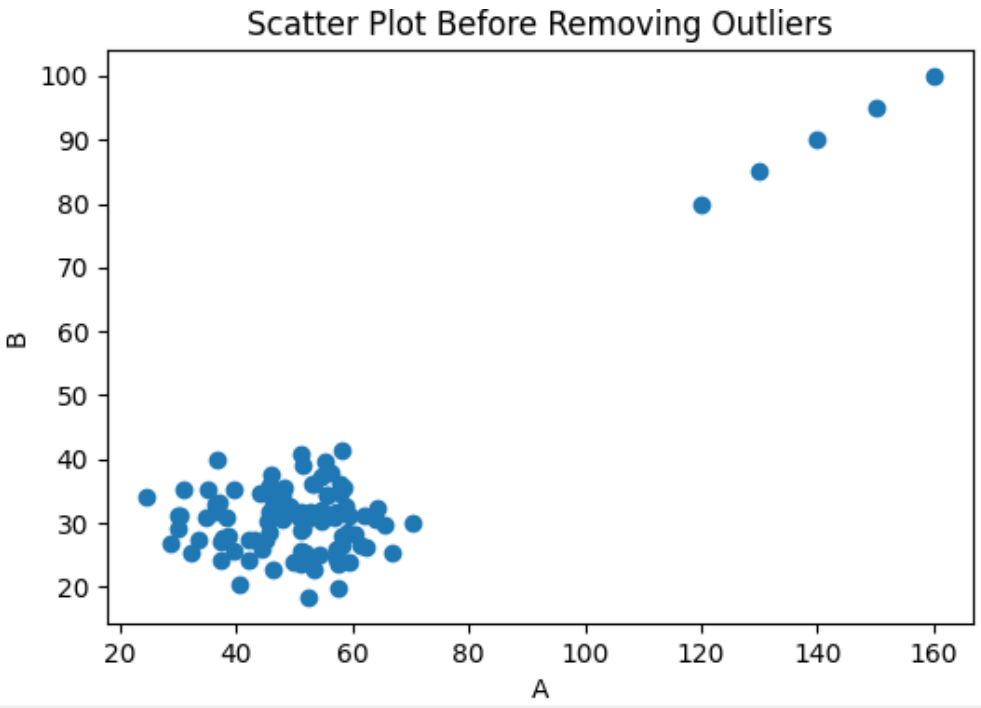
z_scores = np.abs(stats.zscore(df["A"]))
df_z = df[z_scores < 3]
print("After Z-score Removal:", df_z.shape)
```

```
Original Shape: (100, 2)
After IQR Removal: (95, 2)
After Z-score Removal: (96, 2)
```

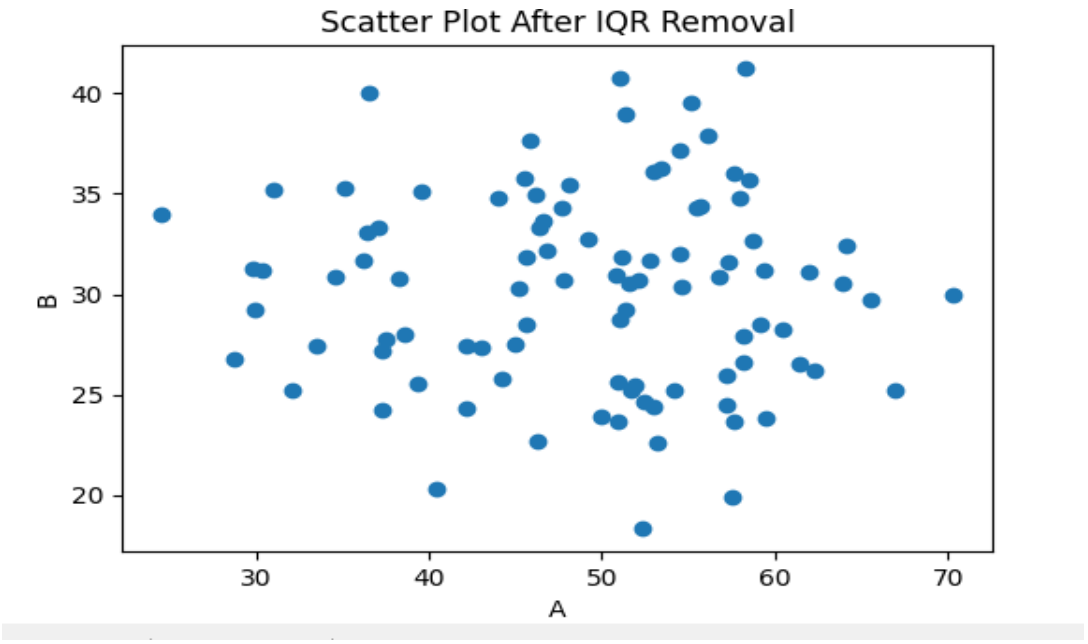
3.scatter plot

```
plt.figure(figsize=(6,4))
plt.scatter(df["A"], df["B"])
plt.title("Scatter Plot Before Removing Outliers")
plt.xlabel("A")
plt.ylabel("B")
plt.show()

plt.figure(figsize=(6,4))
plt.scatter(df_iqr["A"], df_iqr["B"])
plt.title("Scatter Plot After IQR Removal")
plt.xlabel("A")
plt.ylabel("B")
plt.show()
```



ABINYATA T
25MDT1163



4. Grubbs Test

```python
def grubbs_test(x, alpha=0.05):
    N = len(x)
    mean_x = np.mean(x)
    std_x = np.std(x, ddof=1)
    G = max(abs(x - mean_x)) / std_x

    t_critical = stats.t.ppf(1 - alpha/(2*N), N-2)
    G_critical = ((N-1)/np.sqrt(N)) * np.sqrt(t_critical**2 / (N-2 + t_critical**2))

    return G > G_critical

print("Grubbs Test Result for Column A:", grubbs_test(df["A"]))
```

```
Grubbs Test Result for Column A: True
```

ABINYATA T
25MDT1163

5. Mahalanobis outlier detection

```
X = df[["A", "B"]]
cov_matrix = np.cov(X.values.T)
inv_cov_matrix = np.linalg.inv(cov_matrix)
mean = np.mean(X.values, axis=0)

distances = []
for i in range(len(X)):
    dist = mahalanobis(X.iloc[i], mean, inv_cov_matrix)
    distances.append(dist)

df["Mahalanobis_Distance"] = distances

threshold = np.mean(distances) + 3*np.std(distances)
df_maha = df[df["Mahalanobis_Distance"] < threshold]

print("After Mahalanobis Removal:", df_maha.shape)
```

```
After Mahalanobis Removal: (97, 3)
```

6. use scikit-learn for outlier detection

```
iso = IsolationForest(contamination=0.05, random_state=42)
df["iso_outlier"] = iso.fit_predict(df[["A","B"]])
df_iso = df[df["iso_outlier"] == 1]
print("After Isolation Forest:", df_iso.shape)

# ---- Local Outlier Factor ----
lof = LocalOutlierFactor(contamination=0.05)
df["lof_outlier"] = lof.fit_predict(df[["A","B"]])
df_lof = df[df["lof_outlier"] == 1]
print("After LOF:", df_lof.shape)

# ---- One-Class SVM ----
ocsvm = OneClassSVM(nu=0.05)
df["svm_outlier"] = ocsvm.fit_predict(df[["A","B"]])
df_svm = df[df["svm_outlier"] == 1]
print("After One-Class SVM:", df_svm.shape)
```

```
After Isolation Forest: (95, 3)
After LOF: (95, 4)
After One-Class SVM: (92, 5)
```

ABINYATA T
25MDT1163